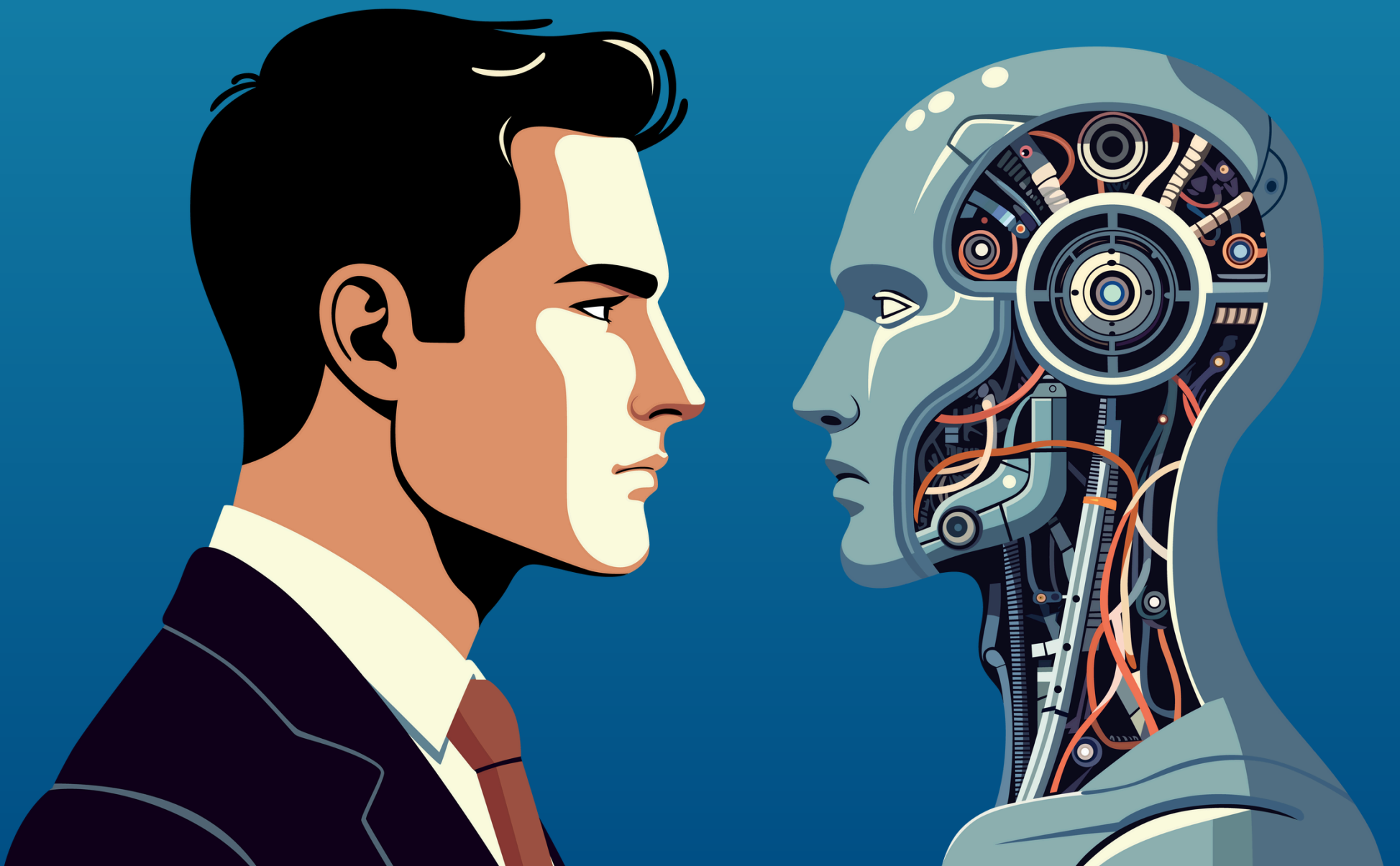




UNDERSTANDING EMBEDDING MODELS

FROM WORD COUNTING
TO
NEURAL REPRESENTATIONS



The Evolution Journey

Traditional Methods (Pre-2013)

- **Bag of Words (BoW):** Simple word frequency counting
- **TF-IDF:** Term frequency weighted by document frequency
- **N-grams:** Capturing word sequences but missing context
- **Co-occurrence Matrices:** Word relationships through statistics
- **Limitations:** Sparse vectors, no semantic understanding, curse of dimensionality

Early Neural Embeddings (2013-2017)

- **Word2Vec:** Skip-gram and CBOW architectures
- **GloVe:** Global vectors for word representation
- **FastText:** Subword information inclusion
- **Doc2Vec:** Document-level representations
- **Breakthrough:** Dense vectors capturing semantic relationships

Transformer Era (2017-Present)

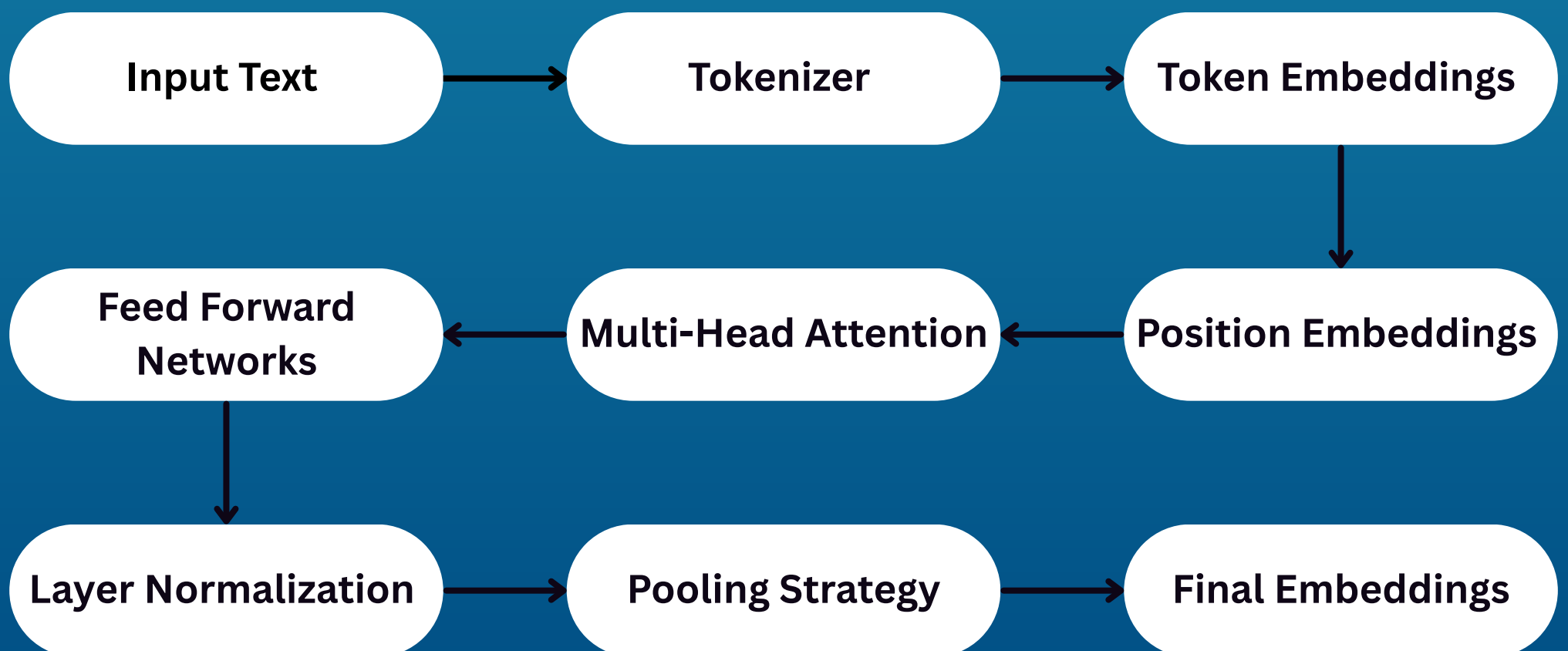
- **Contextual Embeddings:** BERT, RoBERTa, ALBERT
- **Sentence Transformers:** Sentence-BERT, Universal Sentence Encoder
- **Specialized Models:** BGE, E5, Instructor, Ada-002
- **Revolution:** Context-aware, task-specific representations

How Embedding Models Are Created

Training Process

1. **Data Collection:** Massive text corpora (web pages, books, articles, forums)
2. **Preprocessing:** Tokenization, cleaning, deduplication, filtering
3. **Architecture Design:** Transformer encoders with attention mechanisms
4. **Training Objectives:**
 - Contrastive Learning (positive/negative pairs)
 - Masked Language Modeling
 - Multiple Negatives Ranking Loss
 - Cosine Embedding Loss

BEHIND THE SCENES ARCHITECTURE



How Embedding Models Are Created

Key Training Techniques

- **Contrastive Learning:** Learning to distinguish similar vs dissimilar text pairs
- **Hard Negative Mining:** Focusing on challenging negative examples
- **Multi-task Learning:** Training on various tasks simultaneously
- **Knowledge Distillation:** Transferring knowledge from larger models
- **Data Augmentation:** Creating synthetic training pairs

How Modern Embedding Models Work

OpenAI Embedding Models (text-embedding-ada-002)

- **Architecture:** Transformer-based encoder with 12 layers
- **Dimension:** 1536-dimensional vectors
- **Training:** Large-scale contrastive learning on diverse data
- **Specialty:** General-purpose semantic understanding
- **Tokenizer:** BPE with 50k vocabulary

How Modern Embedding Models Work

Hugging Face BGE Models

- **BGE-Large:** 1024 dimensions, 24 layers, 335M parameters
- **BGE-Base:** 768 dimensions, 12 layers, 109M parameters
- **BGE-Small:** 384 dimensions, 12 layers, 33M parameters
- **Training:** Massive Chinese and English corpora with contrastive learning
- **Specialty:** Multilingual retrieval and semantic search

Sentence Transformers (SBERT)

- **Architecture:** BERT/RoBERTa with pooling layers
- **Dimensions:** 384, 768, or 1024 depending on base model
- **Training:** Sentence pair classification and regression
- **Specialty:** Sentence-level semantic similarity

Process Flow

1. **Input Processing:** Text $\rightarrow$ Tokens $\rightarrow$ Input IDs + Attention Masks
2. **Encoding:** Multi-head attention captures token relationships
3. **Pooling Strategies:**
 - CLS token pooling
 - Mean pooling
 - Max pooling
 - Weighted pooling
4. **Normalization:** L2 normalization for cosine similarity

Different Types of Embedding Models

By Architecture

- **Encoder-Only:** BERT-based models (BGE, E5, Sentence-BERT)
- **Dual-Encoder:** Separate encoders for queries and documents
- **Cross-Encoder:** Joint encoding for high accuracy (slower)

By Training Approach

- **Unsupervised:** Word2Vec, GloVe, FastText
- **Self-Supervised:** BERT, RoBERTa with masked language modeling
- **Supervised:** Fine-tuned on labeled similarity data
- **Contrastive:** Trained with positive/negative pairs

By Use Case

- **General Purpose:** OpenAI Ada-002, Universal Sentence Encoder
- **Domain-Specific:** SciBERT, BioBERT, FinBERT
- **Multilingual:** mBERT, XLM-R, BGE-M3
- **Code:** CodeBERT, GraphCodeBERT

Why People Use Embeddings Without Understanding

The Hidden Iceberg

```
# What users see (simple API call)
embeddings = model.encode(["Hello world"])
# Shape: (1, 768) - just numbers

# What's actually happening behind the scenes:
# - Tokenization with 30k+ vocabulary
# - 12+ transformer layers processing
# - 100M+ parameters activated
# - Complex attention patterns computed
# - Months of training on billions of examples
```

The Complexity Behind Simple Vectors

- **Training Time:** Weeks/months on high-end GPUs
- **Data Scale:** Billions of text pairs and examples
- **Parameter Count:** Millions to billions of learned weights
- **Loss Functions:** Complex mathematical objectives
- **Hardware Requirements:** Specialized AI accelerators

From Words to Vectors

The Transformation

Traditional Approach (TF-IDF)

"The cat sits" $\rightarrow$ [0.5, 0.3, 0.8, 0, 0, ..., 0]

- Sparse vector with mostly zeros

Word2Vec Approach

"The" $\rightarrow$ [0.1, -0.3, 0.5, 0.2, -0.1, ...]

"cat" $\rightarrow$ [0.2, 0.1, -0.4, 0.3, 0.6, ...]

"sits" $\rightarrow$ [-0.1, 0.4, 0.1, -0.2, 0.3, ...]

Modern Contextual Embeddings

"The cat sits on the mat"

Each word gets different vectors based on context:

"The" (beginning) $\neq$ "the" (middle) in vector space

Context-aware dense representations

Training Process Deep Dive

Data Preparation

1. **Web Scraping:** CommonCrawl, Wikipedia, Books
2. **Text Cleaning:** Remove HTML, normalize Unicode
3. **Quality Filtering:** Language detection, content filtering
4. **Pair Generation:** Create positive and negative examples

Model Architecture

- **Input Layer:** Token + Position embeddings
- **Transformer Blocks:** Self-attention + feed-forward
- **Pooling Layer:** Convert token $\rightarrow$ sentence embeddings
- **Output Layer:** Normalized dense vectors

Training Loop

1. **Forward Pass:** Process batch of text pairs
2. **Loss Calculation:** Contrastive or triplet loss
3. **Backward Pass:** Gradient computation
4. **Parameter Update:** Adam/AdamW optimization
5. **Validation:** Check performance on held-out data

Applications and Use Cases

Search and Retrieval

- **Semantic Search:** Find documents by meaning, not keywords
- **Question Answering:** Match questions to relevant passages
- **Document Clustering:** Group similar content automatically

Recommendation Systems

- **Content Recommendation:** Suggest similar articles/products
- **User Profiling:** Create user preference vectors
- **Cold Start Problem:** Handle new users/items

Classification and Analysis

- **Text Classification:** Convert text to features for ML
- **Sentiment Analysis:** Capture emotional content
- **Duplicate Detection:** Find similar or identical content

Business Applications

- **Customer Support:** Automatic ticket routing
- **Knowledge Management:** Organize company documents
- **Content Moderation:** Detect inappropriate content

Performance Metrics

Evaluation Methods

- **Cosine Similarity:** Measure vector angle similarity
- **Semantic Textual Similarity (STS):** Human-annotated similarity scores
- **Information Retrieval:** Precision, Recall, NDCG
- **Classification Accuracy:** Downstream task performance

Benchmark Datasets

- **MTEB:** Massive Text Embedding Benchmark
- **SentEval:** Sentence representation evaluation
- **BEIR:** Benchmarking IR with diverse tasks
- **GLUE/SuperGLUE:** General language understanding

Key Takeaways

1. **Evolution:** From sparse word counts to dense neural representations
2. **Complexity:** Simple API calls hide months of sophisticated training
3. **Context Matters:** Modern embeddings understand word meaning in context
4. **Specialization:** Different models excel at different tasks
5. **Foundation:** Embeddings are the backbone of modern NLP applications



FOLLOW US ON



/ STATFUSIONAI